

Build a (RAG) System

Daily Challenge: Build a Retrieval Augmented Generation (RAG) System

What you will create

You will create a functional RAG system that can answer questions based on a dataset loaded from Hugging Face Datasets. This system will:

Load the databricks/databricks-dolly-15k dataset. Index the dataset content into a vector store. Utilize a pre-trained question-answering model from Hugging Face.

Answer user queries by retrieving relevant documents and using the LLM to generate answers.

1. Set up your environment:

```
!pip install -q langchain torch transformers sentence-transformers datasets

import torch
from datasets import load_dataset
from langchain_community.document_loaders import HuggingFaceDatasetLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
from transformers import pipeline
from langchain_huggingface import HuggingFacePipeline
from langchain_classic.chains import RetrievalQA

print("Step 1 Complete: All libraries installed and imported.")
```

```
_____ 23.8/23.8 MB 43.4 MB/s eta 0s
_____ 2.5/2.5 MB 55.6 MB/s eta 0s
_____ 1.0/1.0 MB 26.7 MB/s eta 0s
_____ 543.9/543.9 kB 15.7 MB/s eta 0s
_____ 64.9/64.9 kB 3.3 MB/s eta 0s
_____ 51.0/51.0 kB 1.9 MB/s eta 0s

ERROR: pip's dependency resolver does not currently take into account
google-colab 1.0.0 requires requests==2.32.4, but you have requests 2.
Step 1 Complete: All libraries installed and imported.
```

```
!pip install -q -U langchain-community langchain-huggingface langchain
```

2. Load the dataset:

```
!pip install -Uq datasets
```

```
===== 529.0/529.0 kB 9.0 MB/s eta: 0s  
===== 48.9/48.9 MB 10.7 MB/s eta: 0s
```

```
from langchain_community.document_loaders import HuggingFaceDatasetLoader

dataset_name = "databricks/databricks-dolly-15k"
page_content_column = "context"

loader = HuggingFaceDatasetLoader(dataset_name, page_content_column)

data = loader.load()

print(f"Successfully loaded {len(data)} documents.")
print("\n--- Example Entries ---")
print(data[:2])
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your sett
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to a
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Pleas
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthent
README.md:      8.20k/? [00:00<00:00, 70.5kB/s]

databricks-dolly-                               13.1M/13.1M [00:01<00:00, 65.8MB/s]
15k.jsonl: 100%

Generating train split: 100%                     15011/15011 [00:00<00:00, 85354.12 examples/s]
Successfully loaded 15011 documents.

--- Example Entries ---
[Document(metadata={'instruction': 'When did Virgin Australia start op
```

3. Split the documents:

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk

docs = text_splitter.split_documents(data)

print(f"Original documents: {len(data)}")
print(f"Created {len(docs)} chunks.")
```

```
print("\n--- First Chunk ---")
print(docs[0])
```

Original documents: 15011
Created 18502 chunks.

```
--- First Chunk ---
page_content='\"Virgin Australia, the trading name of Virgin Australia ,
```

4. Embed the text:

```
from langchain_huggingface import HuggingFaceEmbeddings

modelPath = "sentence-transformers/all-MiniLM-L6-v2"
model_kwargs = {'device':'cpu'}
encode_kwargs = {'normalize_embeddings': False}

embeddings = HuggingFaceEmbeddings(
    model_name=modelPath,
    model_kwargs=model_kwargs,
    encode_kwargs=encode_kwargs
)

text = "This is a test document."
query_result = embeddings.embed_query(text)

print("Embeddings model loaded.")
print(f"Sample vector (first 3 numbers): {query_result[:3]}")
```

```

modules.json: 100%                                349/349 [00:00<00:00, 19.9kB/s]
config_sentence_transformers.json: 100%             116/116 [00:00<00:00, 8.30kB/s]
README.md:      10.5k/? [00:00<00:00, 880kB/s]
sentence_bert_config.json: 100%                    53.0/53.0 [00:00<00:00, 4.34kB/s]
config.json: 100%                                  612/612 [00:00<00:00, 53.7kB/s]
model.safetensors: 100%                            90.9M/90.9M [00:01<00:00, 452MB/s]
Loading weights: 100%  103/103 [00:00<00:00, 478.57it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                | Status      | |
-----+-----+---
embeddings.position_ids | UNEXPECTED | |

Notes:
- UNEXPECTED : can be ignored when loading from different task/architecture

tokenizer_config.json: 100%                        350/350 [00:00<00:00, 28.7kB/s]
vocab.txt:         232k/? [00:00<00:00, 5.68MB/s]
tokenizer.json:     466k/? [00:00<00:00, 19.9MB/s]
special_tokens_map.json: 100%                      112/112 [00:00<00:00, 11.0kB/s]
config.json: 100%                                  190/190 [00:00<00:00, 16.5kB/s]
Embeddings model loaded.
Sample vector (first 3 numbers): [-0.03833852335810661, 0.123464651405

```

5. Create a vector store:

```

from langchain_community.vectorstores import FAISS

db = FAISS.from_documents(docs, embeddings)

print("Step 5 Complete: Vector store created and indexed.")

```

Step 5 Complete: Vector store created and indexed.

6. Prepare the LLM model:

```

from transformers import AutoTokenizer, AutoModelForCausalLM, pi
from langchain_huggingface import HuggingFacePipeline

model_id = "gpt2"

tokenizer = AutoTokenizer.from_pretrained(model_id)
tokenizer.pad_token = tokenizer.eos_token
model = AutoModelForCausalLM.from_pretrained(model_id)

```

```

gen_pipeline = pipeline(
    "text-generation",
    model=model,
    tokenizer=tokenizer,
    max_new_tokens=100,
    temperature=0.7,
    device=-1
)

llm = HuggingFacePipeline(pipeline=gen_pipeline)

print("Step 6 Revised: Generation LLM ready.")

```

```

config.json: 100% 665/665 [00:00<00:00, 59.7kB/s]
tokenizer_config.json: 100% 26.0/26.0 [00:00<00:00, 2.16kB/s]
vocab.json: 1.04M/? [00:00<00:00, 19.0MB/s]
merges.txt: 456k/? [00:00<00:00, 12.8MB/s]
tokenizer.json: 1.36M/? [00:00<00:00, 20.8MB/s]
model.safetensors: 100% 548M/548M [00:04<00:00, 2.79GB/s]
Loading weights: 100% 148/148 [00:00<00:00, 488.77it/s, Materializing param=transformer.wte.weight]

GPT2LMHeadModel LOAD REPORT from: gpt2
Key | Status | |
-----+-----+---+
h.{0...11}.attn.bias | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture

generation_config.json: 100% 124/124 [00:00<00:00, 6.54kB/s]
Passing `generation_config` together with generation-related arguments:
Step 6 Revised: Generation LLM ready.

```

7. Build the Retrieval QA Chain:

```

from langchain_classic.chains import RetrievalQA

retriever = db.as_retriever(search_kwargs={"k": 3})

qa = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",
    retriever=retriever,
    return_source_documents=True
)

print("Step 7 Revised: Chain assembled.")

```

Step 7 Revised: Chain assembled.

8. Test your RAG system:

```
question = "What is cheesemaking?"

result = qa.invoke({"query": question})

print("--- RAG Result ---")
print(f"Question: {question}")
print(f"Answer: {result['result']}")

print("\n--- Sources ---")
for doc in result.get("source_documents", []):
    print(f"- {doc.page_content[:150]}...")
```

Both `max\_new\_tokens` (=100) and `max\_length` (=50) seem to have been s

--- RAG Result ---

Question: What is cheesemaking?

Answer: Use the following pieces of context to answer the question at

"The goal of cheese making is to control the spoiling of milk into che

cut using long, blunt knives and 'blocked' (stacked, cut and turned) b

to be ready, the cheese whey must be released. As with many foods the

Question: What is cheesemaking?